



中山大學
SUN YAT-SEN UNIVERSITY

From C to C++

中山大学计算机学院



主讲人：郑贵锋



联系方式：zhenggf@mail.sysu.edu.cn

目录

CONTENTS

01

C++历史

02

“Hello world!”

03

IO流与对象

04

命名空间

05

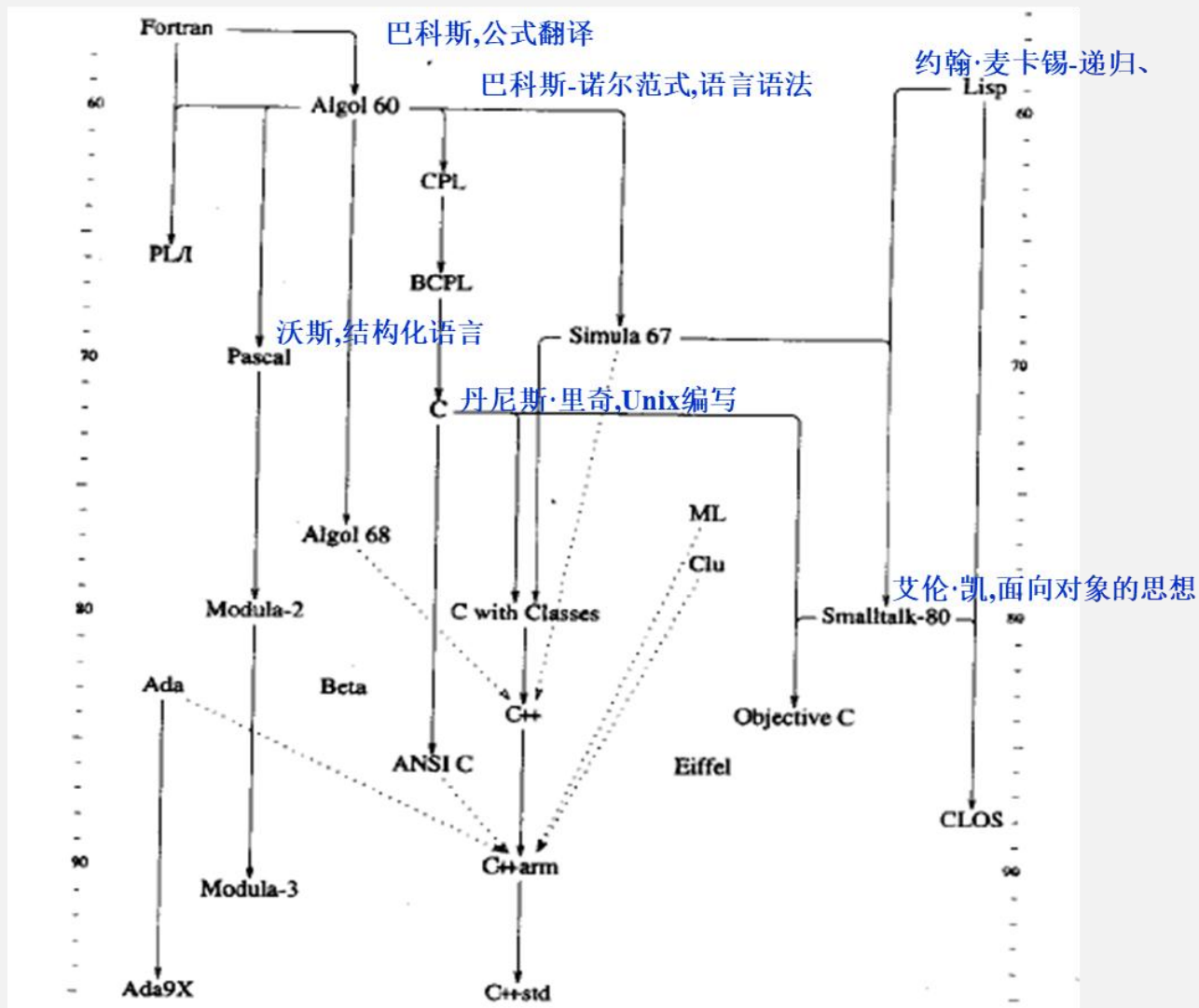
引用 vs 指针

C++历史 - 计算机语言进化图谱

上世纪60's~80's是高级语言创立、创新的年代。

- Fortran 高级语言诞生
- Lisp 递归及符号表达式
- Algol60 巴科斯-诺尔范式
- Pascal 算法+数据=程序
- C 高级语言编写操作系统
- Smalltalk 面向对象思想成熟

期间，各种语言之间**相互借鉴**，C++在这样背景下应运而生。





C++历史 - 面向对象特征简介

C++作为一门面向对象的语言，具有面向对象编程的众多特点：

- 1、封装、继承和多态
 - 2、作为抽象数据类型的类
 - 3、易于调试和维护
 - 4、是软件开发的主流
-

而封装、继承和多态作为面向对象语言的**三大特性**会在后续课程中详细介绍。



C++历史 - C++之父



Bjarne Stroustrup (本贾尼·斯特劳斯特卢普) 在 Bell 实验室时，C 语言性能 (performance) 好，但效率 (productive) 低，萌生了将面向对象语言 *Simula 67* 特征融入 C，并于 1979 年首次实现“带类的 C (C with Classes) ”。



C++历史 - 标准 C++

- Merges notions from **Simula 67** and notions from **C**
- *1979, 1980, C with Classes*, **Bjarne Stroustrup** at **Bell Labs**
- *1983, first C++* compiler implemented
 - Keeps C's efficiency, flexibility and philosophy, while enjoying **object-oriented programming**
- *1985*, Cfront Release 1.0, The C++ programming language V1.0
- *1990*, Cfront Release 3.0, The C++ programming language V2.0
- *1994*, first draft of ANSI/ISO proposed standard
- *1997*, final draft passed, The C++ programming language V3.0
- *1998*, **ANSI/ISO standard, ISO/IEC:98-14882**



C++历史 - 现代 C++

- 2011, C++11标准（实验环境）
- 2014, C++14（测试环境）
- 2017, C++17; ...

Boost库影响很大, 包括正则表达式（[正则表达式详情](#)）、完备的随机数生成函数库、新的时间相关函数, 原子操作支持、标准线程库、新的for语法、auto关键字、新的容器类、更好的union支持、数组初始化列表的支持以及变参模板的支持等



C++ 历史 - 一些新增内容

下面介绍一些C++中和C不同或是新增的语法内容。

- 流式IO操作(`std::cin` , `std::cout`)
- 命名空间
- 引用
- `const` 和 `constexpr`



Hello world C++

```
/* helloworld.cpp */  
#include<iostream>  
using namespace std;  
  
int main() {  
    cout << "hello world c++!" << endl;  
    return 0;  
}
```

- 源代码后缀 cpp 或 cc; 编译工具是 **g++** 不是 gcc
- 标准输入输出库是 iostream, 即没有后缀 ".h" 也不是 "stdio"
- namespace 是 ?
- "cout <<" 是 ? 没有 printf

Dev C++ 设置: Tools - Compiler - Settings - Code Generation - Language standard - ISO C++11



io 流 - 初识对象

- 什么是对象 (Objects) ?
 - 对象是现实世界或抽象世界中事物的一种计算机表示。
 - 例如：猫 tom 在电影《猫和老鼠》中有灰蓝的猫皮，会抓老鼠。
 - 面向对象语言中，声明 `Cat tom`，就可以用 `tom.color` 访问颜色，用 `tom.catch()` 让 tom 执行抓老鼠的行为。
- 定义于头文件 `<iostream>` 有三个对象
 - **cout** 是标准输出流对象，它可使用文件 `stdout` 输出
 - **cin** 是标准输入流对象，它可使用文件 `stdin` 输入
 - **cerr** 是标准错误流对象，它可使用文件 `stderr` 输出



io 流 - 运算符重载

- C++的大多数运算符都可以重载，一句话解释
 - 根据操作数不同，自定义运算符的操作。例如：
 - “>>” 和 “<<” 是位移运算符，它左右操作数必须是整数
- C++形象地将位移运算符用于流对象输入输出操作，例如：
 - `cout << "hello"` 的含义是将 `hello` 字符放入 `stdout` 输出管道；
 - `cin >> aInt` 的含义是将 `stdin` 管道输入按 “%d”格式转换后放入变量；
- 如何区别是文件输入/输出操作还是位移操作？



io 流 - 输入输出运算符

- “<<” 重载为输出运算
 - 左操作数必须是输出流对象
 - 右操作数是任意类型，编译器会根据右操作数类型选择合适的 `fprintf` 格式化语句的版本。如右操作数是 `double`，输出格式是 “%lf”。
 - 表达式返回左操作数的引用，是左值
- “>>” 重载为输入运算
 - 左操作数必须是输入流对象
 - 右操作数必须是左值，任意类型。编译自动根据类型使用合适 `fscanf`
 - 表达式返回左操作数的引用，是左值



io 流 - 输入与输出运算符示例

```
/* iostream-basic.cpp */
#include<iostream>
using namespace std;

int main() {
    int someInt;
    float someFloat;
    char someChar;
    // fscanf(stdin,"%d%f%c", some...);
    cin >> someInt >> someFloat >> someChar;
    // fprintf(stdout,"the answer is: %f\n", some...);
    cout << "the answer is: " << someInt * someFloat << endl;
    return 0;
}
```

- // C++ 风格单行注释
- 运算符重载不会改变运算符的**结合性和优先级**。
例如：
- 位移运算符是左结合，
则 `cin >> someInt` 先计算并返回左值 `cin`，再继续输出 `someFloat`，以此类推
- C 语言是 C++ 基础。
要熟悉表达式的五个要素：类型、值类别、结合性、优先级、类型兼容与隐式转换



io 流 - 输入/输出操纵符 (Manipulators)

- 操纵符是让输入/输出对象控制格式化、或执行文件操作的特殊对象
- 标头<iostream>定义了一些无参的操纵符，包括
 - *endl*, 输出 '`\n`' 并冲洗输出流。例如: `cout << endl;`
 - *dec*, *hex*, *oct*, 更改用于整数输入/输出的基数（进制）
 - *left*, *right*, 设置填充字符的布置，即左对齐或右对齐
 - *fixed*, *scientific*, 更改用于浮点 I/O 的格式化
 - *showpoint*, *noshowpoint*, 控制浮点表示是否始终包含小数点
 - *showpos*, *noshowpos*, 控制是否将 + 号与非负数一同使用
 - ...



io 流 - 输入/输出操纵符 (Manipulators)

- 标头<iomanip>定义了一些有参的操纵符函数，包括
 - *setw(n)*，更改下个输入/输出域的宽度，宽度为 n
 - *setprecision(n)*，更改浮点精度
 - ...



io 流 - 表格输出示例

```
/* iostream-format.cpp */  
#include<iostream>  
#include<iomanip>  
using namespace std;  
  
int main() {  
    double myFloat=123.4578;  
    int myInt=5;  
  
    cout << fixed << showpoint << setprecision(3);  
  
    cout << setw(10) << left << "Float";  
    cout << setw(12) << right << myFloat << endl;  
    cout << setw(10) << left << "Int";  
    cout << setw(12) << right << myInt << endl;  
  
    return 0;  
}
```



命名空间 (Namespace)

- C 语言中，每个应用中全局标识符**能仅能**定义一次。
 - 例如，用户定义了 `fabs`，则会与标库定义冲突。
- C++命名空间提供了一种在大项目中避免名字冲突的方法
 - 在命名空间块内声明的符号被放入一个具名的作用域中，避免这些符号被误认为其他作用域中的同名符号。
 - 多个命名空间块的名字可以相同。这些块中的所有声明在该具名作用域声明。
 - 例如：标准库中符号（类型、变量、常量、函数等）都在 **std** 命名空间块中声明。`cout` 在 `std` 命名空间块中声明，则 `std::cout` 则是该变量的**有限定名**
 - “**::**”是作用域解析运算符。



命名空间 - 声明与使用

- 语法

`namespace 命名空间名 { 声明序列 }`

- 声明具名命名空间

`using namespace 命名空间名 ;`

`using 命名空间名 :: 成员名 ;`

- 从 `using` 指令之后到指令出现的作用域结尾为止，以对任何无限定名字，来自指定命名空间的成员名或指定成员均可见。
- 随着IDE进步，直接指定使用命名空间中的成员更常用。

`命名空间名 :: 成员名`

- 访问限定空间成员



命名空间 - 命名冲突示例

```
/* ns-helloworld.cpp */  
#include<iostream>  
using namespace std;  
  
int main() {  
    const char *cout = "hello world c++!";  
    std::cout << cout << endl;  
    return 0;  
}
```

- using namespace std; 表示可使用 std 命名空间内成员名称的指令
- c字符串指针 cout 隐藏了 std 空间中的定义
- 注意：C++比C严格，const 指针值必须赋值给 const 指针变量
- std::cout 是标准输出流对象**有限定名称**，避免了字符串变量 cout 的命名冲突



命名空间 - 嵌套示例

```
/* ns-embed.cpp */  
#include<iostream>  
using namespace std;  
  
namespace sysu {  
    namespace students {  
        int collegeCount;  
        void printColleges();  
    }  
}  
  
void sysu::students::printColleges() {  
    cout << "Colleges " << collegeCount << endl;  
}
```

- 声明命名空间及其成员
- 定义空间中成员函数

```
int main() {  
    using namespace sysu::students;  
    collegeCount = 23;  
    printColleges();  
    return 0;  
}
```

- 注意两个using作用域不同

```
#include <iostream>
using namespace std;
namespace np1
{
    void func1()
    {
        cout << "np1::func1" << endl;
    }

    namespace np2
    {
        void func1()
        {
            cout << "np1::np2::func1" << endl;
        }
        void func2()
        {
            func1();
        }
    }
}

int main()
{
    np1::np2::func2();
    return 0;
}
```

输出结果为: [填空1]

作答



引用 (Reference)

- 引用的概念
 - 声明具名的变量引用，即**既存对象或函数的别名**。
 - 引用名特点：使用时类似变量，作为参数时能传引用
- 为什么要引入引用语法？
 - 指针变量是“双刃剑”
 - 强大、灵活。可以取地址，解引用，下标、比较、算术运算，指针指针等等
 - 程序容易出BUG，难调试
 - 引用可以减少使用指针甚至完全取代指针
 - 让普通程序员编写可靠、高品质的程序



引用 - 左值声明与示例

- 左值引用的声明
 - `type &别名 [= 左值表达式]`
- 程序说明
 - `x` 和 `*p` 都是 `a` 的引用
 - `x` 使用和普通变量一样
 - 在C++中，尽可能少用指针

```
/* ref-definition.cpp */  
#include<iostream>  
using namespace std;  
  
int main() {  
    int a = 1024;  
    int *p = &a; // p是指针; &a是a的地址  
    int &x = a; // x是引用, 它实际上与a是同一个变量  
    cout << "a = " << a << endl;  
    cout << "x = " << x << endl;  
    cout << "*p = " << *p << endl;  
    x = 2000;  
    cout << "a = " << a << endl;  
    return 0;  
}
```




引用，特别注意！！

引用不是对象；它们不必占用存储，尽管若需要分配存储以实现所需语义（例如，引用类型的非静态数据成员通常会增加类的大小，量为存储内存地址所需），则编译器会这么做。

因为引用不是对象，故

- 不存在引用的数组
- 不存在指向引用的指针
- 不存在引用的引用

更多要掌握的关于引用概念，参见 *cppreference – reference*。包括：左值引用、右值引用、转发引用、悬垂引用等。

面试经常涉及，后面课程可能不再讲解！



引用

例2：在形参中使用引用

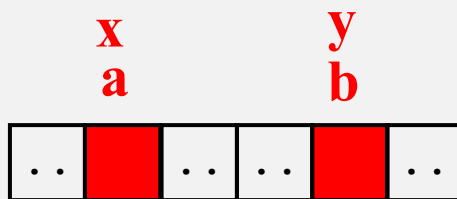
// 程序RefSwap

```
void swap(int &x, int &y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

```
int main()
{
    int a, b;
    cin >> a >> b;

    swap(a, b);

    cout << "max = " << a << " " << "min = " << b;
}
```



tip: c 编译实现采用将a,b的地址作为值传递给函数 swap, 但官方并没有定义引用是值为地址的变量, 就是变量的别名。这样以保证与其他高级语言的引用概念一致



引用

假如形参不使用引用

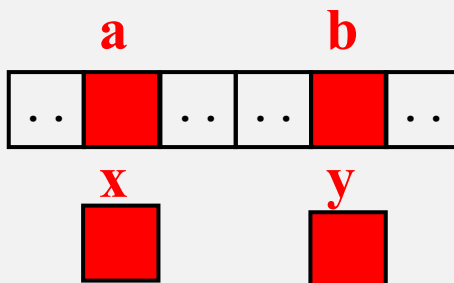
// 程序RefSwap

```
void swap(int x, int y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

```
int main()
{
    int a, b;
    cin >> a >> b;

    swap(a, b);

    cout << "max = " << a << " " << "min = " << b;
}
```





引用

与引用相比，这个指针例子又如何？

```
void swap(int *p1, int *p2)
{
    int temp;

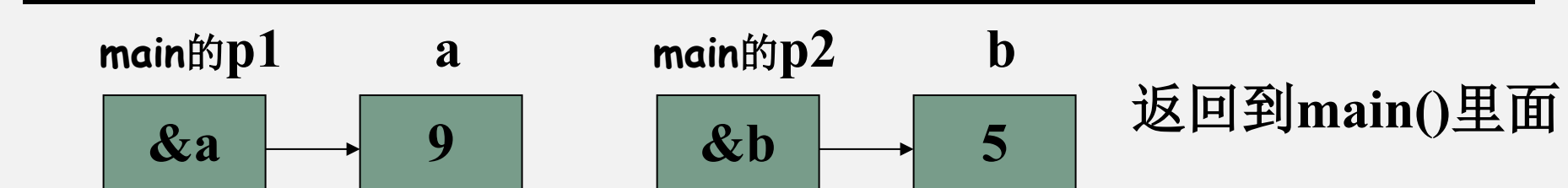
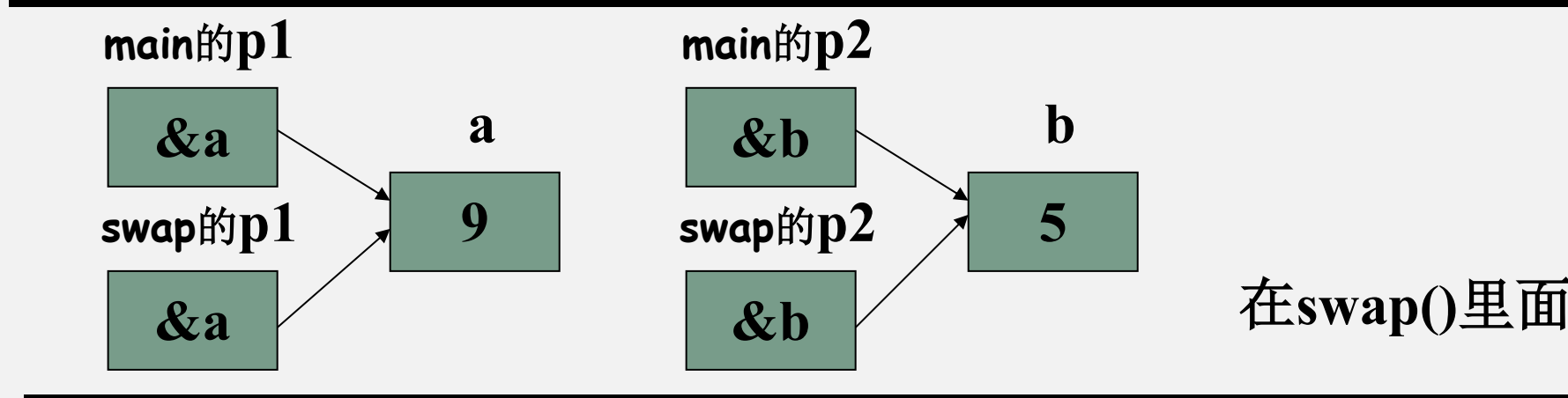
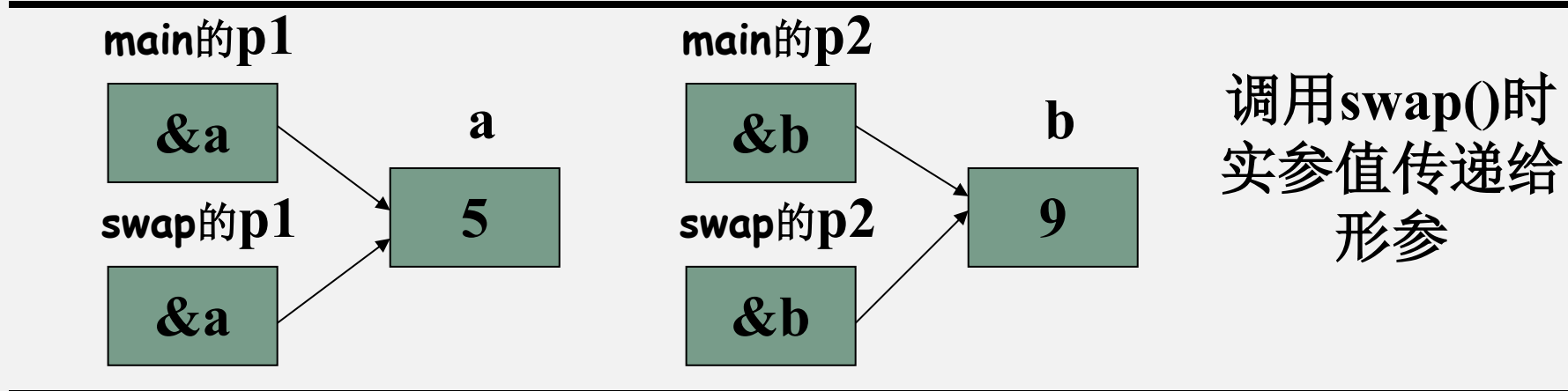
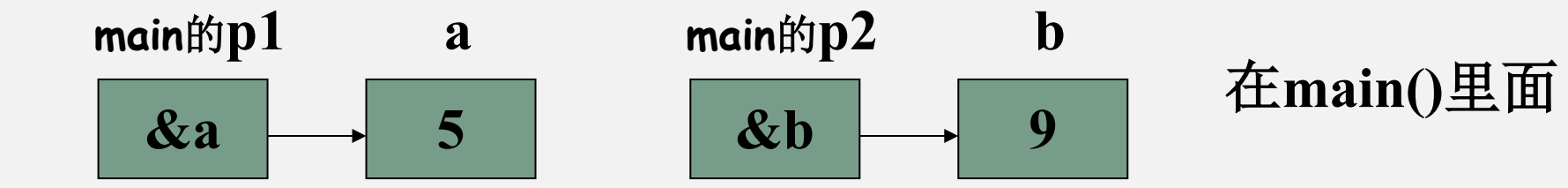
    temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
```

```
int main()
{
    int a = 5, b = 9;
    int *p1, *p2;

    p1 = &a;
    p2 = &b;

    swap(p1, p2);

    cout << "a = " << a << " " << "b = " << b;
}
```





引用

指针与引用

- **相同**：可以使一个函数向调用者**返回多个数值**。
- **不同**：原理不同。引用传递中，形参、实参实质为同一变量，或者说是为某个变量起多了一个名字。而使用指针作函数参数，则是使被调用函数获得某变量的地址，从而使用这个地址访问这个变量。
- 从返回值的角度看，引用形参比利用指针方便。

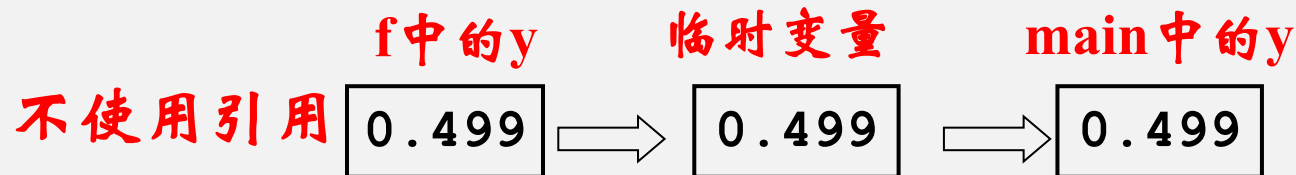


引用

例3 返回值不使用引用

```
double f( double x )  
{  
    double y;  
  
    y = sin(x);  
  
    return y;  
}
```

```
int main()  
{  
    double a = 3.14/6;  
    double y;  
  
    y = f( a );  
  
    cout << "y = " << y << endl;  
  
    return 0;  
}
```





引用

例3 返回值使用引用

```
double& f( double x ) //程序ref2
{
    static double y;

    y = sin(x);

    return y;
}
```

```
int main()
{
    double a = 3.14/6;
    double y;

    y = f( a );

    cout << "y = " << y << endl;

    return 0;
}
```





引用

例3 返回值使用引用

```
double& f( double x )  
{  
    double y;  
  
    y = sin(x);  
  
    return y;  
}
```

```
int main()  
{  
    double a = 3.14/6;  
    double y;  
  
    y = f( a );  
  
    cout << "y = " << y << endl;  
  
    return 0;  
}
```

f 中的 y main 中的 y

使用引用 0.499  0.499

若有说明:

```
int i, j = 6, *p;
```

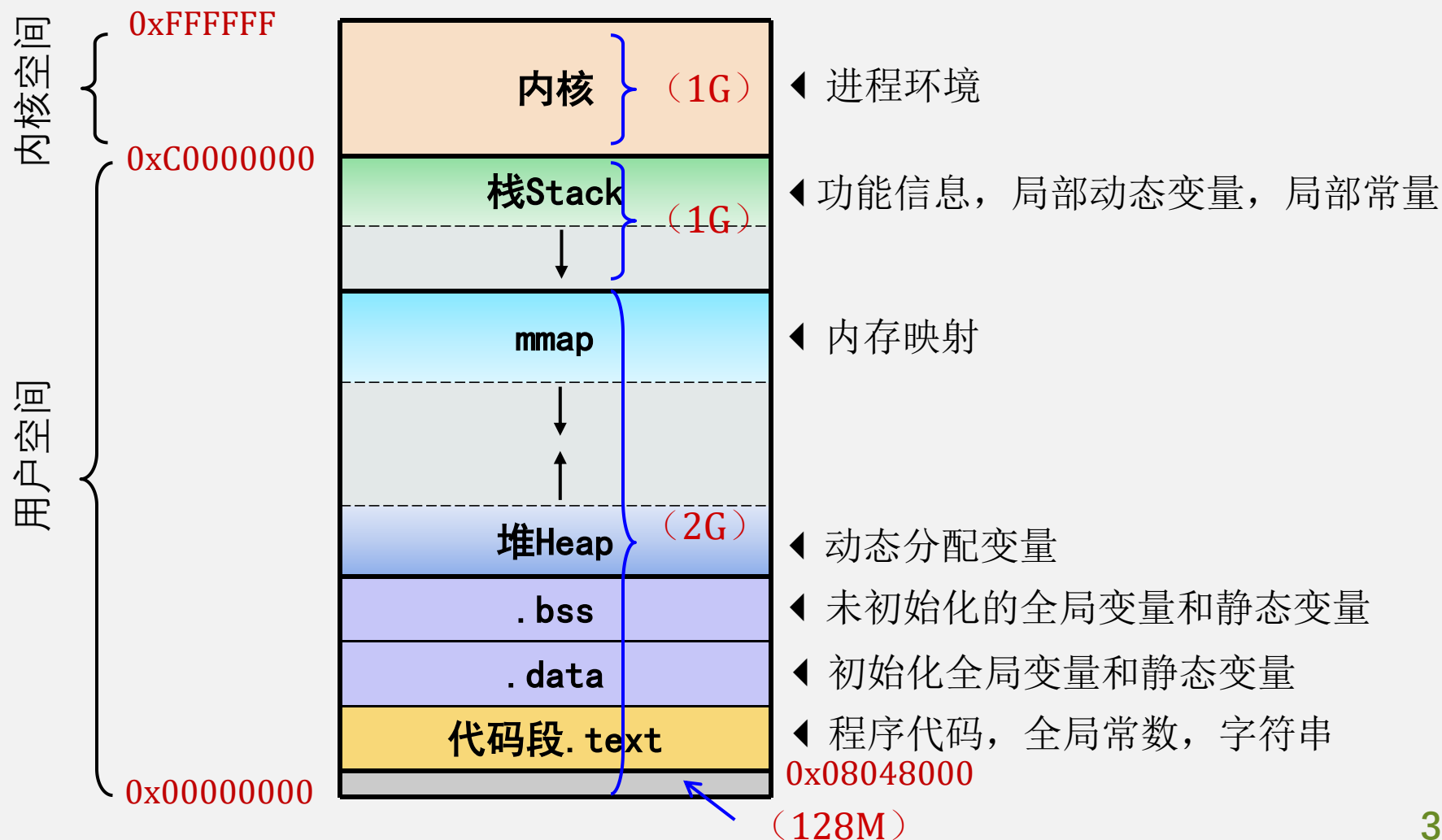
```
p = &i;
```

则与 $i = j$ 等价的语句是()

- ☐ A $i = *p;$
- ☒ B $*p = *&j;$
- ☐ C $i = &j;$
- ☐ D $i = **p;$

提交

补充：C/C++语言内存模型



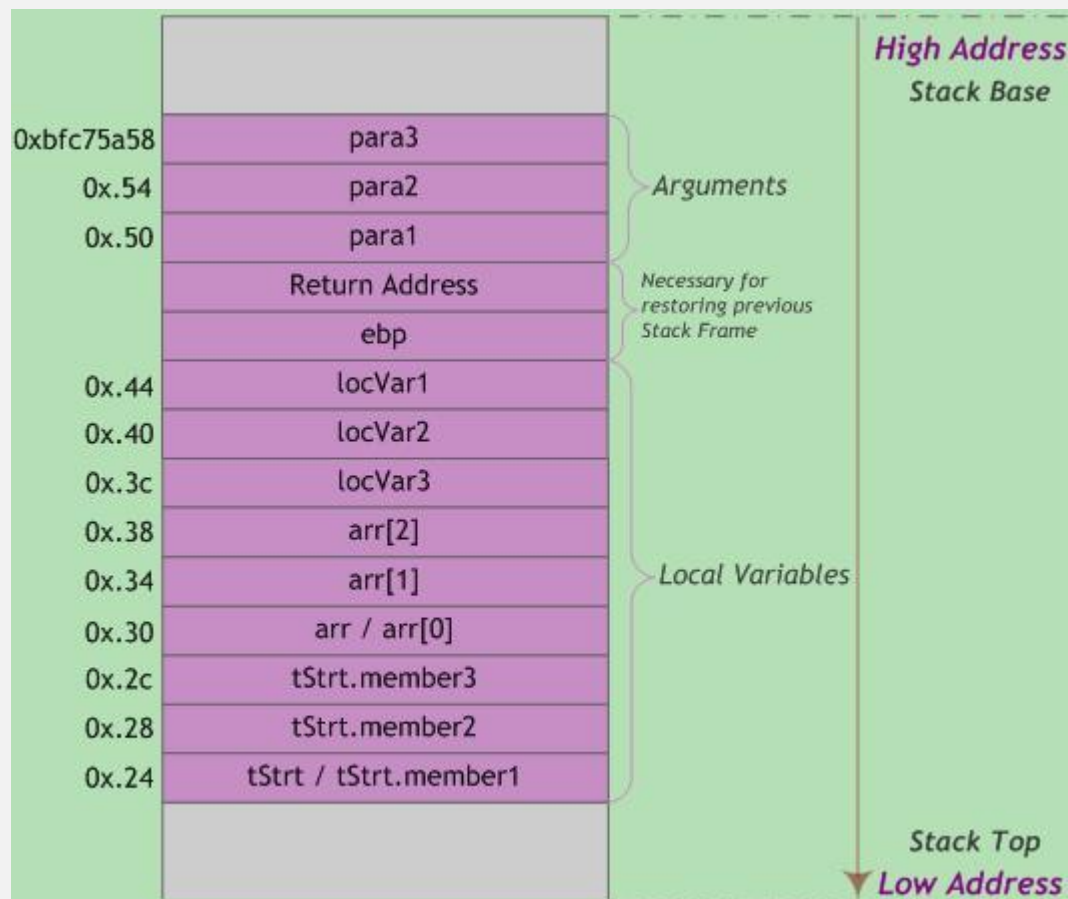


补充：C语言函数调用栈与自动变量

```
struct Strt{
    int member1;
    int member2;
    int member3;
};

int StackFrameContent(int para1, int para2, int para3){
    int locVar1 = 1;
    int locVar2 = 2;
    int locVar3 = 3;
    int arr[] = {0x11,0x22,0x33};
    struct Strt tStrt = {0};
    return 0;
}

int main(void) {
    int locMain1 = 1, locMain2 = 2, locMain3 = 3;
    StackFrameContent(locMain1, locMain2, locMain3);
    return 0;
}
```





中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组